6.3CSC and CSU Descriptions Section

The current computer software configuration item (CSCI) for the Chess Learning Engine is divided into 3 main Computer Software Components (CSC):

- Graphical User Interface CSC (chess board and piece visualization and interaction)
- Chess Logic CSC (chess rules and function for game state and pieces)
- Reinforcement Learning CSC
- Data Storage CSC

6.3.1 Detailed Class Descriptions Section:

The following sections provide the details of all classes used in the Chess Learning Engine application.

6.3.1.1 Class - Piece

Purpose: To store the identity, color, and current position of a piece and provide basic movement templates.

Fields:

- type: String (e.g., "Pawn", "Knight") identifying the piece.
- color: Boolean or Enum (White/Black) defining the piece's affiliation.
- position: Coordinate pair (x, y) representing the current square on the 8x8 grid.

Methods:

- get_position(): Returns the current (x, y) coordinates.
- set_position(new_pos): Updates the piece's location after a validated move.
- Is_white(): Returns the color of the piece
- Get_move() Returns a list of possible moves.

6.3.1.2 Class - File Handler

Purpose: To manage physical disk I/O operations for saving and loading game files.

Fields:

- save_directory: String path where all game files are stored.
- last_accessed_file: String name of the most recently loaded game.

Methods:

- write (data): Serializes the current game state and writes it to a file.
- read (filename): Deserializes a file into a format the GameState class can ingest.

6.3.1.3 Class - Board State

Purpose: To maintain the current layout of the board and track the history of moves and all piece location.

Fields:

- board_array: An 8x8 NumPy array containing references to Piece objects.
- move_log: A list of all moves made in Standard Algebraic Notation (SAN).
- current_turn: Boolean tracking whether it is White or Black's turn to move.

Methods:

- update_board(move): Modifies the board_array after a move is confirmed legal.
- is_king_in_check(): Scans the board to see if the current player's King is under attack.

6.3.1.4 Class – Chess GUI

Purpose: To manage all visual rendering of the chess board and pieces, and to capture and route human player interactions to the logic systems.

Fields:

- display_window: Object representing the main application window, canvas, and UI layout.
- selected_square: Coordinate pair (x, y) tracking which square the user has currently clicked or highlighted.

Methods:

- render_board(board_array): Translates the internal Board State array into visual graphics and sprites on the screen.
- capture_click(): Captures mouse click coordinates, translates them into board squares, and sends the move request to the Board State for validation.
- show_message(text): Displays game status overlays or terminal messages (e.g., "Check", "Game Over", "Engine Thinking...") to the user.

6.3.1.5 Class - Movement Engine

Purpose: This class is responsible for evaluating board positions and selecting the most advantageous move.

Fields:

- max_search_depth: Integer defining how many moves ahead the engine calculates.
- confidence_threshold: Float value determining the minimum score required to select a move.

Methods:

- find_best_move(state): Queries the neural network to identify the optimal move from the current state.
- update_weights(outcome): Triggers the reinforcement learning process to adjust move-valuation weights based on a win or loss.
- evaluate_board(state): Converts current 8x8 board state into a standardized numerical array format.

6.3.1.6 Class: Neural Network

Purpose: To perform high-speed inference on board states and store learned strategic patterns.

Fields:

- model_weights: A multidimensional array (tensor) representing the engine's current "knowledge."
- learning_rate: An integer that dictates how much the weight changes after a game.
- discount_factor: Integer that determines how much the network values immediate rewards (capturing a piece now) versus long-term rewards (achieving checkmate later).

Methods:

- forward_pass(board_vector): Uses a numerical representation of the board and processes it through the network's layers, returning a float value between -1.0 (Black is winning) and 1.0 (White is winning).
- backpropagate(reward_signal, move_history): Triggered at the end of a match. It takes the final game outcome and traces back through the move_history, applying mathematical gradients to update the weights tensor so the engine makes better choices next time.

6.3.2 Detailed Interface Descriptions Section

This section provided an overview of how the components interact.

6.3.2.1 GUI to Move Validation Interface: The GUI request validation of movement to the Move validation containing the starting coordinate, destination coordinate, and piece type.

6.3.2.2 Board-State to Learning Engine Interface: The Board State CSU passes a vectorized representation of the entire board and receives the best action and its confidence value.

6.3.2.3 Learning Engine to Storage Interface: The Learning Engine CSU passes the final game outcome the chronological list of moves, and the updated neural network weights tensor.

6.3.3 Detailed Data Structure Descriptions Section

This section provided an overview of the data structures being used.

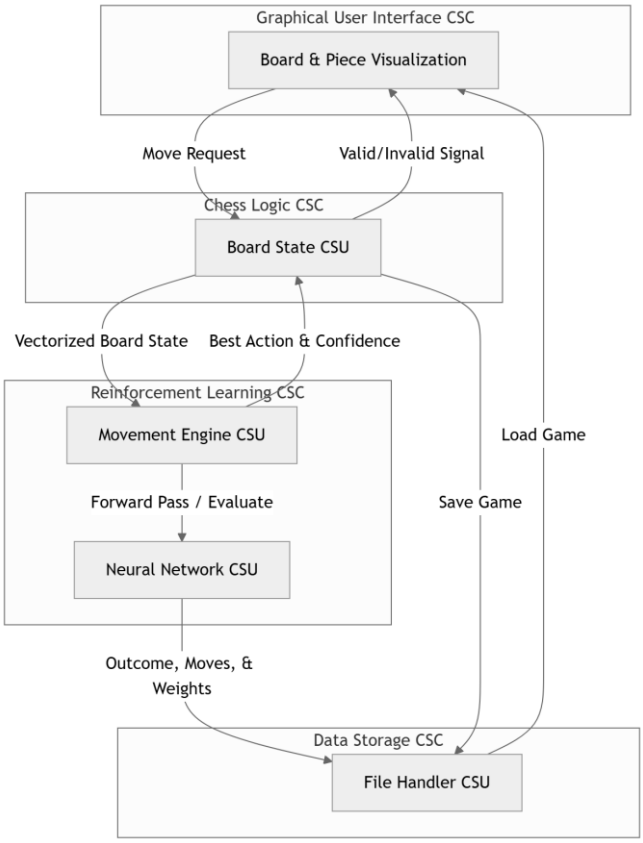
6.3.3.1 Move History Stack: A LIFO (Last-In, First-Out) list containing strings of piece movement and board state vectors.

6.3.3.2 Vectorized Board State: An 8x8x12 numerical tensor representing board and pieces as binary channels.

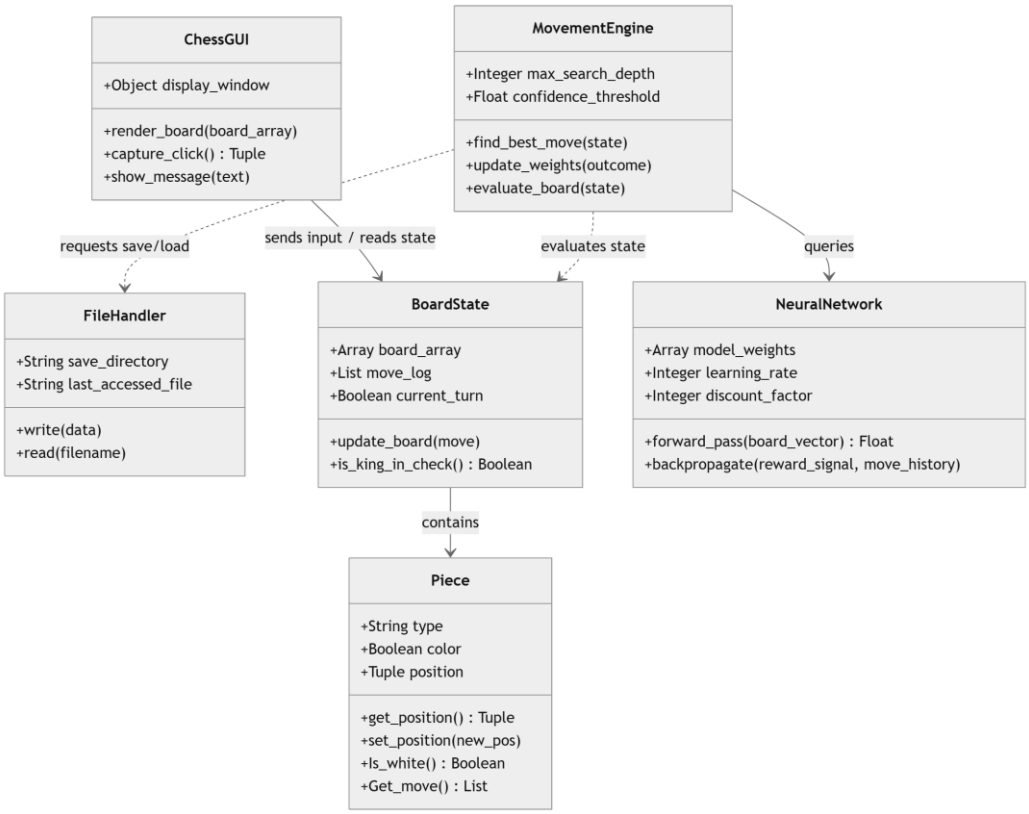
6.3.3.3 Legal Move List: A list of string vectors containing the FIDE-compliant moves for each chess piece.

6.3.4 Detailed Design Diagrams Section

6.3.4.1 Components Diagram:



6.3.4.2 Class Diagram:



6.4Database Design and Description Section

GameSession	
string	GameID
string	Outcome

MoveRecord	
int	MoveID
string	Piece
tuple	PrevCoord
tuple	NewCoord
obj	StateVector

NeuralModel	
string	ModelVersion
float	LearningRate
obj	WeightsTensor

6.4.1

6.4.2 The database will be accessed primarily through the Data Storage CSC, utilizing the File Handler CSU as the central interface for all I/O operations. All CSCs must send their data to the file handler which organizes the information and writes it safely to the disk one step at a time. This prevents the system from crashing or corrupting files if two parts of the program try to save at the same moment. The system will only perform a write action after the game is finished to save the final game results and the AI's new knowledge.

6.4.3 The database is stored locally, so there is minimal point of access. Within the program, the database can only be accessed through one component which reduces the chance of data corruption and errors.